

转换为资源管理对象”两个时间点之间有可能发生异常干扰。

避免这类问题的办法很简单: 使用分离语句, 分别写出 (1) 创建 Widge, (2) 将它置入一个智能指针内, 然后再把那个智能指针传给 processWidget:

```
std::tr1::shared_ptr<Widget> pw(new Widget);    //在单独语句内以
                                                // 智能指针存储
                                                // newed 所得对象。
processWidget(pw, priority());                  //这个调用动作绝不至于造成泄漏。
```

以上之所以行得通, 因为编译器对于“跨越语句的各项操作”没有重新排列的自由 (只有在语句内它才拥有那个自由度)。在上述修订后的代码内, "newWidget" 表达式以及“对 tr1::shared_ptr 构造函数的调用”这两个动作, 和“对 priority 的调用”是分隔开来的, 位于不同语句内, 所以编译器不得在它们之间任意选择执行次序。

请记住

- 以独立语句将 newed 对象存储于 (置入) 智能指针内。如果不这样做, 一旦异常被抛出, 有可能导致难以察觉的资源泄漏。

4

设计与声明

Designs and Declarations

所谓软件设计，是“令软件做出你希望它做的事情”的步骤和做法，通常以颇为一般性的构想开始，最终演变成十足的细节，以允许特殊接口（`interfaces`）的开发。这些接口而后必须转换为 C++ 声明式。本章中我将对良好 C++ 接口的设计和声明发起攻势。我以或许最重要、适合任何接口设计的一个准则作为开端：“让接口容易被正确使用，不容易被误用”。这个准则设立了一个舞台，让其他更专精的准则对付一大范围的题目，包括正确性、高效性、封装性、维护性、延展性，以及协议的一致性。

以下准备的材料并不覆盖你需要知道的优良接口设计的每一件事，但它强调某些最重要的考虑，对某些最频繁出现的错误提出警告，为 `class`、`function` 和 `template` 设计者经常遭遇的问题提供解答。

条款 18：让接口容易被正确使用，不易被误用

Make interfaces easy to use correctly and hard to use incorrectly.

C++ 在接口之海漂浮。`function` 接口、`class` 接口、`template` 接口……每一种接口都是客户与你的代码互动的手段。假设你面对的是一群“讲道理的人”，那些客户企图把事情做好。他们想要正确使用你的接口。这种情况下如果他们对任何一个接口的用法不正确，你至少也得负一部分责任。理想上，如果客户企图使用某个接口却没有获得他所预期的行为，这个代码不该通过编译；如果代码通过了编译，它的作为就该是客户所想要的。

欲开发一个“容易被正确使用，不容易被误用”的接口，首先必须考虑客户可能做出什么样的错误。假设你为一个用来表现日期的 `class` 设计构造函数：